virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Recap: La Rete e l'Effimero

Nel modulo precedente abbiamo visto come l'app "esce" dal telefono per parlare con il mondo tramite le **API** (Retrofit e JSON).
Abbiamo scaricato dati e immagini, gestendo l'asincronia per non bloccare il Main Thread.

Ma c'è un problema.
Cosa succede se scarico 100 messaggi, poi chiudo l'app o vado offline? **Tutto svanisce.**

Obiettivi di Oggi

Dati Locali e Grafica Avanzata

1. **La Gerarchia della Memoria:** Dove salviamo i dati?
2. **Jetpack DataStore:** Salvare le impostazioni (Piccoli dati).
3. **Room Database:** L'architettura dei dati (Grandi dati).
4. **Motion e UX:** Animazioni dichiarative in Compose.
5. **Estetica Avanzata:** Material 3 e Canvas.
6. **Demo:** Un Diario Locale Persistente.

virvelle

Parte 1: Le Memorie dello Smartphone

Architettura dello Storage in Android

L'Amnesia delle Variabili

Tutte le variabili che abbiamo usato finora (le `var`, le `val`, gli `State`, i `rememberSaveable`) vivono nella **RAM** (Random Access Memory).

La RAM è velocissima, ma è **volatile**. Se il sistema operativo uccide l'app o il telefono si spegne, la RAM viene azzerata.
Per far sopravvivere i dati all'infinito, dobbiamo scriverli sullo **Storage Fisico** (Il disco fisso del telefono).

Leggere e Scrivere sul Disco

Scrivere sul disco fisico è un'operazione "pesante".
Ricordate la regola d'oro di ieri?
Non bloccare mai il Main Thread!

Proprio come per la rete, leggere e salvare dati nel disco richiederà l'uso delle **Coroutines** (codice asincrono) per non far laggare l'interfaccia.

Memoria Interna vs Esterna

In Android ci sono due posti principali dove salvare file fisici:

- 🔒 **Storage Interno (App-specific):** Una Sandbox privata. Solo la vostra app può leggere questi file. Se disinstallate l'app, i file vengono distrutti.
- 🔓 **Storage Esterno (Shared):** Cartelle come "Download" o "Immagini". Visibili da tutte le app e dall'utente. Richiedono **Permessi Speciali** per essere letti/scritti.

I 3 Livelli di Persistenza

Cosa vogliamo salvare? La scelta dello strumento dipende dal dato:

1. **Preferenze/Opzioni:** Dati microscopici (es. Tema Scuro attivato? Punteggio record? Utente loggato?). Usiamo **DataStore**.
2. **Dati Strutturati:** Liste giganti, relazioni, ricerche (es. Rubrica, chat, catalogo prodotti). Usiamo **Room (Database)**.
3. **Media:** Foto, video, audio, PDF. Usiamo i **File di Sistema**.

virvelle

Dibattito: Architettura Delle App

Se steste sviluppando **WhatsApp**, dove salvereste queste tre cose?

1. L'impostazione "Nascondi l'ultimo accesso".
2. La foto profilo del vostro amico.
3. Lo storico di tutti i messaggi della chat di classe.

(Pensate agli strumenti visti nella slide precedente)

virvelle

Parte 2: Piccoli Dati, Grandi Soluzioni

Jetpack DataStore

Il Cimitero di SharedPreferences

Fino a pochi anni fa, per salvare piccole impostazioni in Android si usava una classe chiamata `SharedPreferences`.

Era famosa, ma aveva un difetto mortale: spesso leggeva e scriveva sul disco **bloccando il Main Thread**, causando crash improvvisi (ANR) e scatti nell'interfaccia.
Oggi è considerata roba da museo.

Benvenuto Jetpack DataStore

Google ha riscritto tutto da zero creando **DataStore**.
Lavora in modo 100% asincrono, è sicuro, ed è basato sulle Kotlin Coroutines e sui `Flow` (flussi di dati continui).

Si basa sul concetto di **Chiave-Valore (Key-Value)**.

Chiave - Valore (Il Dizionario)

Funziona esattamente come un dizionario o una rubrica:

- **Chiave (La parola):** Un nome univoco (es. "TEMA_SCURO", "PUNTEGGIO_MAX").
- **Valore (La definizione):** Il dato effettivo (es. `true`, `9999`).

Per leggere un dato, non chiedo "dammi il numero 9999", ma dico *"Dammi il valore salvato dentro PUNTEGGIO_MAX"*.

Esempio: Salvare un dato

Così si salva un dato in modo asincrono nel DataStore:

```
// Creiamo la "Chiave" di tipo intero
val PUNTEGGIO_KEY = intPreferencesKey("punteggio_massimo")

// Usiamo una funzione suspend (Coroutine) per scrivere fisicamente
suspend fun salvaPunteggio(nuovoPunteggio: Int) {
    dataStore.edit { preferenze ->
        preferenze[PUNTEGGIO_KEY] = nuovoPunteggio
    }
}
```

Esempio: Leggere un dato (Il Flow)

Leggere un dato da DataStore è "magico". Non ti restituisce il valore e basta. Ti restituisce un **Flow** (Flusso). Se il punteggio cambia domani, il Flow avviserà automaticamente l'interfaccia di aggiornarsi, senza che tu debba richiedere il dato!

Kotlin

```
val punteggioFlow: Flow<Int> = dataStore.data.map { preferenze ->
    // Restituisci il punteggio, o 0 se non esiste ancora
    preferenze[PUNTEGGIO_KEY] ?: 0
}
```

Da Flow a Jetpack Compose

Come colleghiamo questo "tubo" di dati continui alla nostra UI dichiarativa? Con una singola riga di codice: `collectAsState()`.

```
@Composable
fun SchermataProfilo() {
    // La UI si "aggancia" al Flow. Se il file cambia, la UI si ridisegna!
    val punteggio by punteggioFlow.collectAsState(initial = 0)

    Text("Il tuo record storico è: $punteggio")
}
```

Dibattito: Sicurezza dei Dati Locali

DataStore e le vecchie SharedPreferences salvano i dati in formato testo (XML o JSON) dentro le cartelle di sistema.

Se salvate la Password dell'utente dentro DataStore, un utente con il telefono *"Rootato"* (modificato) può aprire quel file di testo e leggere la password in chiaro.

Domanda: *Come facciamo a far fare il login all'utente senza dovergli far digitare la password ogni volta, mantenendo l'app sicura? (Indizio: Token).*

virvelle

Parte 3: Dati Strutturati

L'Arte del Database e Jetpack Room

Quando il Key-Value non basta

Volete creare un'app di Appunti, un Pokedex, o una Rubrica. Avete 1.000 elementi, ognuno con Nome, Descrizione, Foto e Data.

Se provate a salvare 1.000 elementi in DataStore con chiavi tipo `Nota_1`, `Nota_2`... finirete al manicomio. Inoltre, se volete cercare "Tutti gli appunti che contengono la parola Kotlin", DataStore vi obbligherebbe a scorrere tutto il file. Lentissimo!

I Database Relazionali (SQL)

La soluzione a questo problema esiste dagli anni '70: i **Database Relazionali**.

- I dati sono salvati in **Tabelle** (Righe e Colonne).
- Si interrogano i dati usando un linguaggio specifico chiamato **SQL** (Structured Query Language).

In Android, il motore database ufficiale integrato in ogni telefono si chiama **SQLite** (un database intero salvato in un singolo file).

Il problema di SQLite "Puro"

Lavorare direttamente con SQLite su Android richiede di scrivere tonnellate di codice verboso ("Boilerplate"). Bisogna aprire connessioni, usare cursori, gestire gli errori e convertire manualmente le righe di testo in Oggetti Kotlin.

Un errore di battitura nel codice SQL e l'app crasha al volo durante l'uso.

Benvenuto, Room!

In One Piece, Trafalgar Law crea una sfera magica chiamata *"Room"*. All'interno di quello spazio, lui ha il controllo assoluto e istantaneo sulla materia, potendo smontare e ricomporre le cose senza fatica.

Noi useremo la libreria **Jetpack Room**, che fa esattamente la stessa cosa con il nostro database SQLite!

Il Potere di Jetpack Room

Room è una libreria di astrazione. Crea uno spazio (una *Room*) dove noi programmatori parliamo solo in puro **Kotlin**. Room, dietro le quinte, traduce la nostra magia Kotlin in query SQL complesse.

- Se facciamo un errore di sintassi, l'app **non compila** (ci avvisa prima di crashare).
- Si integra perfettamente con le Coroutines.

L'Architettura: I 3 Pilastri di Room

Per creare un database con Room ci servono 3 mattoncini:

1. **Entity (La Tabella):** Una classe Kotlin che rappresenta lo stampo dei nostri dati (es. Il singolo Appunto).
2. **DAO (Il Telecomando):** Le funzioni per manipolare i dati (Insert, Delete, Search).
3. **Database:** La classe centrale che unisce Entity e DAO e crea fisicamente il file nel telefono.

Pilastro 1: L'Entity

Con Room, non dobbiamo scrivere codice SQL per creare tabelle. Ci basta usare le "Annotazioni" (`@`) su una normale `data class` .

Kotlin

```
@Entity(tableName = "tabella_appunti")
data class Appunto(
    // Questo è il nostro ID univoco autogenerato!
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,

    @ColumnInfo(name = "titolo_nota")
    val titolo: String,

    val contenuto: String
)
```

virvelle

Pilastro 2: Il DAO (Data Access Object)

Il DAO è l'elenco delle mosse segrete che possiamo usare dentro la nostra "Room". Room scriverà il codice noioso per noi!

Kotlin

```
@Dao
interface AppuntiDao {
    @Insert
    suspend fun inserisciAppunto(appunto: Appunto)

    @Delete
    suspend fun eliminaAppunto(appunto: Appunto)

    // Qui scriviamo noi la query SQL personalizzata!
    @Query("SELECT * FROM tabella_appunti ORDER BY id DESC")
    fun leggiTuttiGliAppunti(): Flow<List<Appunto>>
}
```

L'osservazione perfetta

Avete notato che `leggiTuttiGliAppunti()` restituisce un `Flow`? Room e Compose sono migliori amici.

Se mostrate a schermo la lista degli appunti tramite quel Flow, e in background scaricate una nuova nota aggiungendola al DB... l'interfaccia si aggiornerà da sola con l'animazione, senza che voi dobbiate dirgli di ridisegnare!

Pilastro 3: Il Database

Infine, dichiariamo la stanza principale che unisce tabelle e DAO.

```
@Database(entities = [Appunto::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

    // Il ponte per accedere al nostro telecomando!
    abstract fun appuntiDao(): AppuntiDao
}
```

Dibattito: Locale vs Cloud

Avete appena speso una settimana a progettare un database Room perfetto. La vostra app di Note è bellissima.

Un vostro utente perde il telefono. Ne compra uno nuovo, installa la vostra app e... il database è vuoto. Tutti gli appunti di una vita sono persi.

Cosa manca a questa architettura? Come potremmo unire il Modulo 4 (Rete) al Modulo 5 (Database Room) per creare un'app invincibile? (Indizio: Single Source of Truth).

virvelle

Parte 4: Grafica Avanzata e Motion UX

Oltre la staticità

L'Importanza dell'Animazione

Nelle interfacce moderne, l'animazione non serve solo a fare "scena". Serve a far capire all'utente cosa sta succedendo (Motion UX).

- Se un elemento sparisce di colpo (Snap), il cervello si confonde.
- Se l'elemento rimpicciolisce svanendo, il cervello comprende che è stato cestinato.

In Jetpack Compose, le animazioni sono **State-Driven** (guidate dallo Stato).

AnimatedVisibility

Volete far apparire un messaggio di errore solo quando `haErrore` è `true`? Invece di usare un semplice `if`, usiamo `AnimatedVisibility`.

```
var visibile by remember { mutableStateOf(false) }

AnimatedVisibility(
    visible = visibile,
    enter = fadeIn() + expandVertically(),
    exit = fadeOut() + shrinkVertically()
) {
    Text("Sono apparso con una transizione fluida!")
}
```

animateContentSize

Un altro trucco magico di Compose. Se un componente cambia dimensione (es. una Card con un testo lungo che viene espanso), basta un `Modifier` per rendere l'espansione animata invece che immediata.

```
Card(
    modifier = Modifier
        .fillMaxWidth()
        .animateContentSize(
            animationSpec = tween(durationMillis = 500)
        )
) {
    // Contenuto che può espandersi
}
```

Le Curve di Easing (La Fisica)

Nella realtà, le cose non si muovono a velocità costante. Un'auto accelera e frena. In Compose usiamo gli **Easing** per simulare la fisica e dare naturalezza.

- `LinearEasing`: Costante, innaturale, meccanico.
- `FastOutSlowIn`: (Standard di Material) Parte veloce, frena dolcemente. Elegante.
- `Bounce`: Rimbalza alla fine. Molto giocoso.

Lottie: L'arma segreta dei Designer

Per animazioni complesse (un omino che corre, un fuoco d'artificio, un checkmark 3D) non le scriviamo a mano col codice!

L'industria usa **Lottie** (creato da Airbnb). I designer animano su After Effects, esportano in formato **JSON**, e noi carichiamo quel file di testo nella nostra app. Avremo animazioni leggerissime a 60fps in 3 righe di codice.

virvelle

Parte 5: Estetica Finale

Material 3 e Canvas

Material You (Colori Dinamici)

Con Android 12 e Material Design 3, Google ha introdotto i colori dinamici. Non scegliete più un colore fisso (es. "L'app è Verde").

Jetpack Compose può estrarre i colori dallo Sfondo del telefono dell'utente e applicarli ai vostri bottoni automaticamente. La vostra app diventerà l'app "Personale" dell'utente.

Il Canvas: Disegno Libero

Se `Row` e `Column` non bastano perché dovete disegnare un grafico a torta, un radar o un videogioco, Compose offre il `Canvas`.

Kotlin

```
Canvas(modifier = Modifier.fillMaxSize()) {
    // Coordinate libere per dipingere coi pixel!
    drawCircle(
        color = Color.Blue,
        radius = 100f,
        center = Offset(size.width / 2, size.height / 2)
    )
}
```

Mettiamoci alla prova!

Kahoot Time 🕹️

Vediamo chi ha capito la differenza tra DataStore, SQLite e le "Room" di Trafalgar Law!

virvelle

Kahoot Time!

PIN: 0687019

Breve Demo 💻

(Condivido lo schermo: Analizziamo il codice del Diario Locale e vediamo Room in azione!)

virvelle

Fase 1: Scaffolding

(Vediamo insieme la demo di oggi)

L'Architettura del Progetto

I mattoni del Database (Room)

Il codice che vi ho passato è diviso in file separati, proprio come lavorano i professionisti.

1. **`Nota` (L'Entity):** È lo "stampo" dei nostri dati. Guardate `@PrimaryKey(autoGenerate = true)`: non dobbiamo inventarci noi gli ID, Room conta da solo (1, 2, 3...).
2. **`AppDatabase`:** È la "cassaforte". Usa un pattern chiamato *Singleton* (`INSTANCE`). Serve a garantire che ci sia una sola connessione al database aperta alla volta, altrimenti il telefono va in crash.
3. **`NotaDao`:** *(Sottinteso nel codice)* È il telecomando con i tasti `insert`, `delete` e `getAllNotes`.

Il Ponte: Il ViewModel

Perché c'è un file `MainViewModel`?
Ricordate la regola d'oro: **Non bloccare il Main Thread!**

Il ViewModel fa il lavoro sporco:

- Prende il testo dalla UI.
- Apre una **Coroutine** asincrona (`viewModelScope.launch`).
- Salva nel database in background.
- Restituisce i dati alla UI sotto forma di **Flow** (`val note: Flow<List<Nota>>`). Se il DB cambia, la UI si aggiorna da sola per magia.

virvelle

Sfida Autonoma

(Tocca a voi estendere il Diario)

Il vostro turno: Estendere l'App

Scegliete il vostro obiettivo o provate a completarli tutti e tre:

TASK	TIPO	DESCRIZIONE
1. Motion UX	🎨 Grafica	Aggiungi `Modifier.animateItem()` alla LazyColumn per rendere fluido l'inserimento e l'eliminazione delle note.
2. Data Evolution	⚙️ Logica	Aggiungi il campo `isUrgente` (Boolean) nell'Entity. Se vero, mostra un'icona 🔴 nella Card della nota.
3. Crash Test	💥 Debug	Rimuovi il `viewModelScope.launch` dal salvataggio e osserva il crash nel Logcat (Main Thread violation).

⚠️ Attenzione: Schema Change!

Se modificate l'**Entity** (Task 2), l'app andrà in crash all'avvio perché il database esistente non corrisponde più al nuovo codice.

Soluzione rapida:

1. Disinstalla l'app dall'emulatore/telefono (Long press sull'icona -> Uninstall).
2. Riesegui il progetto da Android Studio.
3. Il database verrà ricreato da zero con i nuovi campi.

Promemoria sull'IA 🤖

Usate ChatGPT o Gemini come **Copiloti**, non come scorciatoie.

❌ *"Scrivimi un'app meteo intera in Jetpack Compose"* (Non imparerete nulla).
✅ *"Come faccio a passare dei parametri dinamici (latitudine e longitudine) a un LaunchedEffect quando premo un bottone?"* (Questo è fare Ingegneria del Software).

Buon lavoro!

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary